COM759: Knowledge Engineering & AI

15 Multiple Choice Questions on Propositional Logic, Automated Reasoning & Prolog (Week 3)

Question 1: Trade-off in Knowledge Representation Languages

In formal logical systems used for Knowledge Representation, how are **Effectiveness** and **Efficiency** defined and balanced?

A) Effectiveness measures memory space, while Efficiency measures syntactic correctness.

B) Effectiveness corresponds to expressiveness (measured by completeness and correctness), while Efficiency corresponds to computational complexity (measured by time and space consumption).

C) Effectiveness applies only to First-Order Logic, whereas Efficiency applies only to Propositional Logic.

D) Effectiveness determines proof theory execution speed, while Efficiency determines semantic validity.

Correct Answer: B — Effectiveness corresponds to expressiveness, while Efficiency corresponds to computational complexity.

Explanation: In logical KR language design, Effectiveness relates to expressiveness (adequacy to accomplish a purpose, measured by completeness and correctness), whereas Efficiency relates to computational complexity (measured by time and space resource consumption). A fundamental engineering trade-off exists between expressiveness and complexity.

Question 2: Formal System Architecture

Which three formal specifications rigorously define any **Formal Logic System** in computer science?

A) Entities, Attributes, and Relationships.

B) Well-defined Syntax, Well-defined Semantics, and Well-defined Proof Theory.

C) Data Structure, Search Algorithm, and Heuristic Function.

D) Fact Base, Rule Base, and User Interface.

Correct Answer: B — Well-defined Syntax, Well-defined Semantics, and Well-defined Proof Theory.

Explanation: Any formal logical system consists of: Syntax (defining syntactically acceptable well-formed formulae / wffs), Semantics (associating formulae with meanings or world models), and Proof Theory (axioms and inference rules for deducing sentences from a Knowledge Base).

Question 3: Scope and Abstraction of Propositional Logic

What is the defining structural characteristic of **Propositional Logic (PL)** compared to First-Order Logic (FOL)?

A) PL evaluates internal object structures, predicates, and quantifiers over domains.

B) PL asserts propositions that are either true or false, dealing strictly with complete statements without evaluating relationships or dependencies between individual objects.

C) PL assigns continuous probabilistic truth values between 0 and 1 to non-declarative statements.

D) PL restricts all propositions to Horn clauses containing single literals.

> **Correct Answer: B — PL asserts complete propositions without evaluating internal object relationships.**
>
> **Explanation:** Propositional Logic is the simplest and most abstract formal logic. It asserts complete propositions (declarative statements evaluating to either True or False) and does not represent or evaluate internal object properties or dependencies between objects.

Question 4: Formalizing Natural Language Constructs

Consider the English statement: *"Carlo comes to the party provided that Davide doesn't come, but, if Davide comes, then Bruno doesn't come."* Using propositional symbols *C*, *D*, and *B*, what is the exact formalization?

A) (C ¬D) (D ¬B)

B) (D ¬C) (B ¬D)

C) (C ¬D) (D ¬B)

D) ¬(C D) B

> **Correct Answer: A — (C ¬D) (D ¬B)**
>
> **Explanation:** "Carlo comes provided that Davide doesn't come" translates to (C ¬D), and "if Davide comes, then Bruno doesn't come" translates to (D ¬B). The conjunction "but" joins them with , yielding (C ¬D) (D ¬B).

Question 5: Semantics of Material Implication ()

In propositional semantics, why does the material implication formula $P \ Q$ evaluate to **True** when the antecedent P is **False**?

A) Because false antecedents imply a causal failure that defaults to True.

B) Because material implication is defined as P Q ¬P Q; when P is false, no contractual promise is broken, so the formula evaluates to True by default.

C) Because false premises automatically make any conditional formula a valid tautology.

D) Because propositional truth tables invert false inputs across logical connectives.

> **Correct Answer: B — Defined as P Q ¬P Q; when P is false, the condition holds by default.**
>
> **Explanation:** Material implication P Q does not assert a causal link. It is logically equivalent to ¬P Q. When P is False, the antecedent condition was not met (no obligation broken), so the statement P Q evaluates to True by default.

Question 6: Semantic Taxonomies: Validity, Satisfiability, and Unsatisfiability

How are **Valid**, **Satisfiable**, and **Unsatisfiable** propositional formulas formally defined?

A) Valid formulas are true in at least one model; Satisfiable formulas are true in all models; Unsatisfiable formulas are false in at least one model.

B) Valid formulas are satisfied by every interpretation (tautologies); Satisfiable formulas are satisfied by at least one interpretation; Unsatisfiable formulas are satisfied by no interpretation (contradictions).

C) Valid formulas contain no negations; Satisfiable formulas are in CNF; Unsatisfiable formulas contain Horn clauses.

D) Valid formulas are decidable in linear time; Satisfiable formulas are NP-complete; Unsatisfiable formulas are non-deterministic.

> **Correct Answer: B — Valid = satisfied by all models; Satisfiable = satisfied by ≥1 model; Unsatisfiable = 0 models.**
>
> **Explanation:** A formula F is Valid (tautology) if every interpretation satisfies it; Satisfiable if at least one interpretation satisfies it; and Unsatisfiable (inconsistent/contradictory) if no interpretation satisfies it. All valid formulas are satisfiable, while unsatisfiable formulas are not satisfiable.

Question 7: Truth Table Complexity and the SAT Problem

For a propositional formula containing *n* distinct atomic proposition symbols, how many rows exist in its complete truth table, and what is the computational complexity status of checking formula validity/satisfiability?

A) n^2 rows; solvable in deterministic polynomial time $O(n^2)$.

B) 2^n rows; whether a polynomial-time algorithm exists to check validity/satisfiability remains a famous open problem in computer science (P vs. NP).

C) n! rows; proven to be algorithmically undecidable.

D) 2n rows; solvable in linear time using unit resolution.

Correct Answer: B — 2^n rows; P vs. NP open problem.

Explanation: Since each atomic symbol can take 2 truth values (True or False), a truth table for n atomic propositions contains 2^n distinct interpretation rows. Finding a polynomial-time algorithm for checking formula validity/satisfiability is a famous open problem (P vs. NP).

Question 8: Canonical Normal Forms: Conjunctive Normal Form (CNF)

What is the formal structural definition of a propositional formula in **Conjunctive Normal Form (CNF)**?

A) A disjunction of conjunctions of literals (l_{ij}).

B) A conjunction of clauses, where each clause is a disjunction of literals ($\bigwedge_{i=1}^{m} (\bigvee_{j=1}^{n_i} l_{ij})$).

C) A chain of material implications ending in an atomic literal.

D) A single clause containing no negated proposition symbols.

Correct Answer: B — A conjunction of clauses, where each clause is a disjunction of literals.

Explanation: A literal is an atomic proposition or its negation (P, ¬P). A clause is a disjunction of literals (S R ¬P). CNF is defined as a conjunction of clauses: $(l_{11} \dots l_{1n_1}) \dots (l_{m1} \dots l_{mn_m})$.

Question 9: Algorithmic Transformation to CNF

When converting the formula ¬*(P (Q R))* into Conjunctive Normal Form (CNF), what is the resulting set of clauses?

A) {P Q, ¬R}

B) {P, ¬Q ¬R}

C) {¬P Q, ¬P R}

D) {P ¬Q ¬R}

> **Correct Answer: B — {P, ¬Q ¬R}**
>
> **Explanation:** Applying equivalence rules step-by-step: 1. Eliminate implication: ¬(¬P (Q R)). 2. Apply De Morgan's Law: ¬¬P ¬(Q R). 3. Eliminate double negation & apply De Morgan's: P (¬Q ¬R). This yields two CNF clauses: {P} and {¬Q ¬R}.

Question 10: SAT Solver Data Formats (DIMACS CNF)

In the standard **DIMACS CNF** input text format accepted by industrial SAT solvers like MiniSAT, how are variables and clauses encoded?

A) Variables are string identifiers, and clauses are grouped inside parentheses separated by AND.

B) Each clause is represented on a line of space-separated integers where positive numbers are variables, negative numbers are negated variables, and each clause line terminates with a 0.

C) Clauses are formatted as XML trees with and tags.

D) Variables are assigned binary bitmasks terminated by a semicolon.

> **Correct Answer: B — Space-separated integers (positive/negative literals) terminated by a 0 on each clause line.**
>
> **Explanation:** DIMACS CNF uses line-based integer formatting. Non-comment lines define clauses as space-separated integers (4 means x_4, -5 means $\neg x_5$), and every clause line must terminate with a trailing space and 0.

Question 11: Metatheory: Soundness vs. Completeness

In formal logical deduction, how are **Soundness** and **Completeness** formally distinguished for a proof system relative to a Knowledge Base (*KB*)?

A) Soundness means KB Q KB Q; Completeness means KB Q KB Q.

B) Soundness means KB Q KB Q (only true entailments are derived); Completeness means KB Q KB Q (all entailed formulas can be proved).

C) Soundness applies to truth tables; Completeness applies only to resolution.

D) Soundness guarantees polynomial execution time; Completeness guarantees zero memory overhead.

Correct Answer: B — Soundness: KB Q KB Q; Completeness: KB Q KB Q.

Explanation: Soundness (KB Q KB Q) ensures that an inference engine produces only valid logical entailments without generating contradictions. Completeness (KB Q KB Q) ensures that the inference rules are capable of deriving every formula that is logically entailed by the KB.

Question 12: The Resolution Inference Rule Mechanics

What is the mechanical operation of the **Resolution** inference rule when applied to two clauses containing complementary literals?

A) Given A B and ¬B C, derive A C.

B) Given A B and ¬B C, eliminate the complementary pair (B, ¬B) to derive the resolvent clause A C.

C) Given A B and B C, derive A ¬C.

D) Given A B and B C, merge them into A B C.

Correct Answer: B — Eliminates the complementary literal pair (B, ¬B) to yield resolvent clause A C.

Explanation: Resolution is a sound inference rule that takes two clauses containing complementary literals (e.g., B and ¬B) and cuts the complementary pair to produce a new resolvent clause implied by the parents: (A B, ¬B C) / (A C).

Question 13: Proof by Contradiction via Refutation Resolution

How does automated reasoning execute a **Proof by Contradiction (Refutation)** to prove that *KB Q*?

A) It converts KB {Q} into CNF and proves it is valid.

B) It negates the query Q, adds ¬Q to the KB, converts KB {¬Q} into CNF clauses, and repeatedly applies resolution until deriving the empty clause (), establishing unsatisfiability.

C) It evaluates truth table models until Q produces a false evaluation.

D) It derives Modus Ponens iterations until KB is empty.

Correct Answer: B — Negates Q, adds ¬Q to KB, converts to CNF, and resolves until deriving the empty clause ().

Explanation: KB Q is valid if and only if KB ¬Q is unsatisfiable. A refutation resolution system adds the negated query ¬Q to KB, converts the set to CNF, and uses resolution to derive the empty clause (), proving unsatisfiability.

Question 14: Evaluating Models of a Knowledge Base

Given a Knowledge Base *KB = [P Q R, Q P]*, which of the following truth assignments *(P, Q, R)* represents an **invalid model** (does NOT satisfy the KB)?

A) (F, F, F)

B) (T, F, F)

C) (F, T, F)

D) (T, T, T)

Correct Answer: C — (F, T, F) evaluates Q P to False, failing to satisfy the KB.

Explanation: Evaluating (F, T, F) across the KB rules: Q P T F False. Since one formula in KB evaluates to False, (F, T, F) fails to satisfy the KB and is not a valid model.

Question 15: Logic Programming Mechanics in Prolog

What class of logical clauses forms the foundation of **Prolog**, and what computational mechanisms govern its inference and search procedures?

A) Disjunctive Normal Form clauses evaluated via forward chaining algorithms.

B) Horn clauses (at most one positive literal) evaluated via resolution inference and depth-first backtracking search with unification.

C) First-Order Quantified clauses evaluated via breadth-first search without variables.

D) Probabilistic logic rules evaluated via linear programming matrices.

Correct Answer: B — Horn clauses evaluated via resolution and depth-first backtracking with unification.

Explanation: Prolog programs consist of sets of Horn clauses (clauses with at most one positive literal, written as $H \leftarrow B_1, ..., B_n$). Inference is driven by resolution, and search is executed using depth-first backtracking with unification.